

Questions for the benchmark

- Does reusing vetted code improve accuracy or reduce the cost of a complete forecasting workflow?
- Our skills teach agents how to use library APIs from Python. weather-skills primarily exposes parameterized commands.
- Working hypothesis: agents need Python to recover from data, parameter, and workflow errors.
- Compare task success, tokens, cost, and completion time across model sizes. Do not assume skills will win.

Current experiment: three execution conditions

- **Python only:** the agent writes, runs, and revises its own code using standard scientific Python packages.
- **Skills only:** the agent reads guides and invokes weather-skills operations; custom Python is disabled.
- **Python + skills:** the agent can combine catalog operations with its own Python. Skill use is measured, not assumed.
- Ten models, one task, three conditions: 30 planned attempts. The small-model tier includes 1B, 3B, 7B, 8B, and 9B models.

Astra is excluded because it helped develop the benchmark. Earlier runs remain archived.

Current task: a two-week Kenya heat outlook

- Use a real, archived ECMWF S2S forecast to calculate the hottest daily regional-mean temperature in each week.
- Required order: cosine-latitude spatial mean for each member/day → weekly peak per member → ensemble median, spread, minimum, and maximum.
- Pass requires a structured answer matching an independent numeric reference, correct units and source citation, and a valid, nonblank figure.
- Every attempt gets the same cached raw data and an empty workspace. Intermediate results stay within that attempt; no outputs or history cross runs.

Equal ceilings: 24 model calls, 40 executions, 200k tokens, 600 seconds.

Download and setup are excluded so solve time measures analysis, not data transfer.

Weather agent benchmark

Heat outlook · 10 models · Astra excluded · Partial · snapshot

Experiment

Heat outlook · 10 models · Astra excluded · 17/30 · Partial · snapshot ▼

Model

All models ▼

Task

All experiment tasks ▼

17 recorded attempts · 17 task outcomes · 10 models · 1 tasks · 1 repetition · ≥\$2.596 reported spend · billing incomplete

Results Tasks 1 Runs 17

GPT-6 Astra is excluded from this comparison because it helped develop the benchmark, tasks, and harness. This avoids evaluating the benchmark-authoring model; it does not establish that the remaining evaluation is free of design bias. Earlier Astra runs remain in archived experiments.

17 / 30 attempts recorded Partial · snapshot · Updated 11:35 AM



Unfinished experiment. Pending cells have no result.

Model / task coverage	Kenya heat outlook · frozen real forecast		
Claude Fable 5.1	Python only Pass	Skills only Pass	Python + skills Pass
Claude Sonnet 5.5	Python only Pass	Skills only Fail	Python + skills Pass
Gemini 3.1 Flash-Lite	Python only Pass	Skills only Fail	Python + skills Pass
DeepSeek V4.1 Flash	Python only Pass	Skills only Pass	Python + skills Pass
Qwen3.5 9B	Python only Pending	Skills only Fail	Python + skills Fail

Interim snapshot: 17/30 attempts. Select a result to inspect its trace.

Observed failure modes

- **Skills bypassed:** 0 of 6 Python + skills attempts invoked a skill. The prompt makes skills optional; this condition tests availability, not actual reuse.
- **Interface friction:** wrong action names, XML tool-call wrappers, guide-read requirements, and invalid CLI arguments consumed turns.
- **Recovery and delivery:** Python indexing and unit errors required retries. Sonnet produced a figure but failed the strict artifact-submission contract.
- **Budget exhaustion:** 6 of 7 failures hit the token limit; Qwen's hybrid run hit the time limit. Repeated context and unusable responses contributed.

Initial six-model batch: 18 attempts, 7 failures. One attempt per model and condition.

Next: test the libraries and their skill guides directly

- **Baseline:** general scientific Python; the agent implements the workflow itself.
- **Libraries:** the same Python environment plus acmadDL and africaS2S, with ordinary API documentation.
- **Libraries + skills:** the same libraries plus agent guides and progressively disclosed workflow references.
- Keep Python available in every condition. Use seasonal tasks covering data acquisition, downscaling, held-out verification, and forecast delivery.

Predeclare tasks and scoring; freeze versions and seeds; repeat across regions, methods, and models.